

Provable Security

Lecturer: Kathrin Hövelmanns

Lecture 11

Recap

KEMs and Fujisaki-Okamoto

KEM = Public-key transport of symmetric keys:

- $\text{Encaps}(pk)$: outputs a ciphertext $c \in \mathcal{C}$ and a key $K \in \mathcal{K}$.
- $\text{Decaps}(sk, c)$: outputs a key $K \in \mathcal{K}$.

IND-CPA (CCA): Keys look random (even against chosen-ciphertext attacks).

Fujisaki-Okamoto builds IND-CCA-secure KEMs from PKE:

- Encaps: Pick $m \leftarrow \mathcal{M}$, set $c \leftarrow \text{Enc}(pk, m)$, $K := H(m, c)$
- Decaps: Set $m' := \text{Dec}(sk, c)$ and $K := H(m', c)$

OW of PKE gives CPA of FO-based KEMs (lossy):

Honest challenge key $K_0^* := H(m^*, c^*)$ is pseudorandom in the RO, unless you query H on m^* .

Then we can use $\mathcal{A}$ to invert c^* .

Games $\text{IND-CPA/CCA}_{\text{KEM}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$b \xleftarrow{\$} \{0,1\}$

$(K_0^*, c^*) \leftarrow \text{Encaps}(pk)$

$K_1^* \xleftarrow{\$} \mathcal{K}$

return pk, c^*, K_b^*

Decaps($c \neq c^*$):

return $K := \text{Decaps}(sk, c)$

Finalize ($b' \in \{0,1\}$):

return $b' =_? b$

Recap

KEMs and Fujisaki-Okamoto

Fujisaki-Okamoto:

- Encaps: Pick $m \leftarrow \mathcal{M}$, set $c \leftarrow \text{Enc}(\text{pk}, m)$, $K := H(m, c)$
- Decaps: Set $m' := \text{Dec}(\text{sk}, c)$ and $K := H(m', c)$

Issues when we tried to show “OW also gives CCA”:

We need to simulate Decaps, which proved tricky:

- Attempt 1: Simulate H and Decaps by plain lazy sampling.
- Problem: “Everything too random” - neither Decaps nor H notice when they should use the same K .
- Attempt 2: Have H notice when c decrypts to m , ensure consistency with Decaps.
- Problem: “Where did the bouncer go?” Our simulation gives a key for every ciphertext, even invalid ones.
- Attempt 3: Have Decaps notice invalid cts and reject.
- Problem: “Plain OW” reductions can’t simulate this.

Games IND-CPA/ $\text{CCA}^{\mathcal{A}}_{\text{KEM}}$

Initialize:

$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}$

$b \xleftarrow{\$} \{0,1\}$

$(K_0^*, c^*) \leftarrow \text{Encaps}(\text{pk})$

$K_1^* \xleftarrow{\$} \mathcal{K}$

return pk, c^*, K_b^*

Decaps($c \neq c^*$):

return $K := \text{Decaps}(\text{sk}, c)$

Finalize ($b' \in \{0,1\}$):

return $b' =_? b$

Recap

KEMs and Fujisaki-Okamoto

To show CCA, we made a stronger requirement on PKE:

One-Wayness in the presence of oracles **PtC**heck and **V**alid, OW-PCVA.

Reminder $\text{KEM}^\perp = U^\perp[\text{PKE}, H]$

Encaps(pk):

$m \xleftarrow{\$} \mathcal{M}$
 $c \leftarrow \text{Enc}(pk, m)$
 $K := H(m, c)$
return (K, c)

Decaps(sk, c):

$m' := \text{Dec}(sk, c)$
if $m' = \perp$
 return $\perp$
else
 return $K := H(m', c)$

• Simulating this without
 sk needed **Valid**

• Simulating this without
 sk needed **PTcheck**

Game $\text{OWPCVA}_{\text{PKE}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $c^* \leftarrow \text{Enc}(pk, m^*)$
return pk, c^*

PTcheck($m \in \mathcal{M}, c \in \mathcal{C}$):

$m' := \text{Dec}(sk, c)$
return $m' =_? m$

Valid($c \in \mathcal{C}$):

$m' := \text{Dec}(sk, c)$
If $m' \in \mathcal{M}$ return true
else return false

Finalize($m' \in \mathcal{M}$):
return $m' =_? m^*$

Recap

KEMs and Fujisaki-Okamoto

To show CCA, we made a stronger requirement on PKE:

One-*W*ayness in the presence of oracles ***PtC*heck** and ***V*alid**, OW-PCVA.

This is somewhat unsatisfactory:

- OW-PCVA is stronger and less natural than OW.
- Fewer PKEs qualify, and proving OW-PCVA can be tedious.

Now: Two ways to enable IND-CCA without requiring full OW-PCVA from PKE:

1. **Augment the transform itself so that PKE's security requirement no longer involves *V*alid.**
2. Show that for deterministic PKEs, PKE's security requirement does not need to involve *PtC*heck.

Game $\text{OWPCVA}_{\text{PKE}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

return pk, c^*

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

*PtC*heck ($m \in \mathcal{M}, c \in \mathcal{C}$)

$m' := \text{Dec}(sk, c)$

return $m' =_? m$

*V*alid ($c \in \mathcal{C}$)

$m' := \text{Dec}(sk, c)$

If $m' \in \mathcal{M}$ return true

else return false

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

We first augment the U-transform – we change how Decaps responds to invalid ciphertexts.

Definition U-transform with implicit rejection

Let $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a PKE scheme with message space $\mathcal{M}$, and let $H: \mathcal{M} \times \mathcal{C} \rightarrow \{0,1\}^n$ be a hash function. We define KEM:

<u>KeyGen(pk):</u> $(pk', sk') \leftarrow \text{KeyGen}$ $s \xleftarrow{\$} \mathcal{M}$ $sk := (sk', s)$ return (pk', sk)	<u>Encaps(pk):</u> $m \xleftarrow{\$} \mathcal{M}$ $c \leftarrow \text{Enc}(pk, m)$ $K := H(m, c)$ return (K, c)	<u>Decaps(sk, c):</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $K := H(s, c)$ else return $K := H(m', c)$
--	--	--

This PKE-to-KEM transform is called the $U^\perp$ **transform**, and the resulting KEM is denoted as $\text{KEM}^\perp$ or $U^\perp[\text{PKE}, H]$.

The treatment of failing ciphertexts is called **implicit rejection**.

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

We formalize the security property of PKE on which we will base the security of $\text{KEM}^\perp$: **One-Way** security in the presence of oracle **PTcheck**.
(Like OW-PCVA, except that $\mathcal{A}$ does not get Valid oracle.)

Definition OW-PCA security

We define the **OW-PCA advantage** of an adversary $\mathcal{A}$ against PKE as

$$\text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{A}) := \Pr[\text{OWPCA}_{\text{PKE}}^{\mathcal{A}} \Rightarrow 1].$$

Game $\text{OWPCA}_{\text{PKE}}^{\mathcal{A}}$

<u>Initialize:</u>	<u>PTcheck</u> (m, c)
$(pk, sk) \leftarrow \text{KeyGen}$	$m' := \text{Dec}(sk, c)$
$m^* \xleftarrow{\$} \mathcal{M}$	return $m' =_? m$
$c^* \leftarrow \text{Enc}(pk, m^*)$	
return pk, c^*	

Finalize ($m' \in \mathcal{M}$):
return $m' =_? m^*$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim $(\text{PKE OW-PCA} \Rightarrow \text{KEM}^{\perp} \text{IND-CCA})$

For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}^{\perp}$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^{\perp}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Main idea:

Only difference to proof of $\text{KEM}^{\perp}$: How we simulate **Decaps** for the new rejection method.

Intuition: since s is secret, responses $K := H(s, c)$ look like well-formed keys.

If $\mathcal{A}$ can't notice which c get rejected, we don't need oracle Valid.

Game $\text{IND-CCA}_{\text{KEM}^{\perp}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$s \xleftarrow{\$} \mathcal{M}$

$b \xleftarrow{\$} \{0,1\}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

$K_0^* := H(m^*, c^*)$

$K_1^* \xleftarrow{\$} \mathcal{K}$

return pk, c^*, K_b^*

Finalize ($b' \in \{0,1\}$):

return $b' =_? b$

$H(m, c)$:

if $\exists K$ such that $(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Decaps($c \neq c^*$):

$m' := \text{Dec}(sk, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim $(\text{PKE OW-PCA} \Rightarrow \text{KEM}^{\not\downarrow} \text{IND-CCA})$

For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}^{\not\downarrow}$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^{\not\downarrow}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions:

G_0 = IND-CPA game for $\text{KEM}^{\not\downarrow}$.

G_1 : Abort if $\mathcal{A}$ ever finds a key belonging to an invalid c , so if H gets queried on s, c for such a c .

G_2 : Replace Decaps, H by to-be-specified simulations that work without oracle VAL.

G_3 : Abort if CHALL happens, so if H gets queried on m^*, c^* .

Game $\text{IND-CCA}_{\text{KEM}^{\not\downarrow}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$s \xleftarrow{\$} \mathcal{M}$

$b \xleftarrow{\$} \{0,1\}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

$K_0^* := H(m^*, c^*)$

$K_1^* \xleftarrow{\$} \mathcal{K}$

return pk, c^*, K_b^*

Finalize ($b' \in \{0,1\}$):

return $b' =_? b$

$H(m, c)$:

if $\exists K$ such that

$(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Decaps ($c \neq c^*$):

$m' := \text{Dec}(sk, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim (PKE OW-PCA $\Rightarrow$ KEM $\not\sim$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $\not\sim$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM} \not\sim}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H / |\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions:

G_0 = IND-CPA game for KEM $\not\sim$.

G_1 : Abort if $\mathcal{A}$ ever finds a key belonging to an invalid c , so if H gets queried on s, c for such a c .

G_2 : Replace Decaps, H by to-be-specified simulations that work without oracle VAL.

G_3 : Abort if CHALL happens, so if H gets queried on m^*, c^* .

Game IND-CCA $^{\mathcal{A}}$
KEM $\not\sim$

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps($c \neq c^*$):

$m' := \text{Dec}(\text{sk}, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

$H(m, c)$:

if $\exists K$ such that
 $(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim (PKE OW-PCA $\Rightarrow$ KEM $\not\sim$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $\not\sim$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM} \not\sim}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H / |\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions: G_0 to G_1

G_0 = IND-CPA game for KEM $\not\sim$.

G_1 : Abort if $\mathcal{A}$ ever finds a key belonging to an invalid c , so if H gets queried on s, c for such a c .

G_2 : Replace Decaps, H by to-be-specified simulations that work without oracle VAL.

G_3 : Abort if CHALL happens, so if H gets queried on m^*, c^* .

Games G_0, G_1

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps ($c \neq c^*$):

$m' := \text{Dec}(\text{sk}, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

$H(m, c)$:

if $m = s$

and query was by $\mathcal{A}$

BAD := true

Abort

if $\exists K$ such that

$(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim (PKE OW-PCA $\Rightarrow$ KEM $\not\Downarrow$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $\not\Downarrow$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM} \not\Downarrow}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions: G_0 to G_1

G_0 = IND-CPA game for KEM $\not\Downarrow$.

G_1 : Abort if $\mathcal{A}$ ever finds a key belonging to an invalid c , so if H gets queried on s, c for such a c .

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\text{BAD}].$$

$\mathcal{A}$'s view is indep. of $s \xleftarrow{\$} \mathcal{M}$ unless G_1 aborts, so

$$\Pr[\text{BAD}] \leq q_H/|\mathcal{M}|.$$

Games G_0, G_1

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps ($c \neq c^*$):

$m' := \text{Dec}(\text{sk}, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

$H(m, c)$:

if $m = s$

and query was by $\mathcal{A}$

BAD := true

Abort

if $\exists K$ such that

$(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle in PKE req

Security claim (PKE OW-PCA $\Rightarrow$ KEM $^\perp$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $^\perp$,
issuing at most q_H queries to H , there exists an
OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions: G_1 to G_2

G_1 : Abort if $\mathcal{A}$ ever finds a key belonging to an
invalid c , so if H gets queried on s, c for such a c .

G_2 : Replace Decaps, H by simulations that work
without oracle VAL (like the ones in G_1 in the proof
for KEM $^\perp$, just without VAL).

We want to show

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G_2^{\mathcal{A}} \Rightarrow 1].$$

Game G_2

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps($c \neq c^*$):

if $\exists K$ such that (c, K)
 $\in T_D$

return K

$K \xleftarrow{\$} \mathcal{K}$

$T_D := T_D \cup \{(c, K)\}$
return K

$H(m, c)$:

if $m = s$

and query was by $\mathcal{A}$

BAD := true

Abort

Lazy sample.

If $m \in M$:

ensure Decaps
consistency via
PTcheck(m, c) and
table T_D
(like G_1 in the proof
for KEM $^\perp$)

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle

We want to show

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G_2^{\mathcal{A}} \Rightarrow 1].$$

We only need to show that the code in both games does the same.

We fix any c , set $m' := \text{Dec}(\text{sk}, c)$, and study cases:

- Case 1: $m' = \perp$ or $m' = s$.

Game 1: Decaps returns $K := H(s, c)$.

Game 2: Decaps finds no $(c, K) \in T_D^*$, and outputs (and stores) a fresh uniform K .

In both games: K is fresh, uniform and - unless BAD triggers an abort - not used anywhere else.

* $\exists (c, K) \in T_D$ would need that H already put it there.

Which H only does when queried on $(m \in M, c)$ such that $m = \text{Dec}(\text{sk}, c)$, so $m = m'$. But $m' = \perp$ or $m' = s$.

Original code in game G_1

Decaps($c \neq c^*$):

$m' := \text{Dec}(\text{sk}, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

H(m, c):

(Abort if BAD)

if $\exists K$ such that

$(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Changed code in game G_2

Decaps($c \neq c^*$):

if $\exists K$ such that (c, K)

$\in T_D$

return K

$K \xleftarrow{\$} \mathcal{K}$

$T_D := T_D \cup \{(c, K)\}$

return K

H(m, c):

(Abort if BAD)

Lazy sample.

If $m \in M$: ensure

Decaps consistency

via $\text{PTcheck}(m, c)$

and table T_D

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle

We want to show

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G_2^{\mathcal{A}} \Rightarrow 1].$$

We only need to show that the code in both games does the same.

We fix any c , set $m' := \text{Dec}(\text{sk}, c)$, and study cases:

- Case 2: $m' \neq \perp$ and $m' \neq s$.

Game 1: Decaps returns $K := H(m', c)$.

Game 2: Decaps also returns $K = H(m', c)$:

If Decaps is queried before $H(m', c)$: Decaps finds no $(c, K) \in T_D$. Returns (and stores in T_D) a fresh uniform K . $H(m', c)$ will find that K in T_D , due to $\text{PTcheck}(m, c)$.

If $H(m', c)$ is queried before Decaps: Returns (and stores in T_D due to $\text{PTcheck}(m, c)$) a fresh uniform K . Decaps will find that K in T_D and return it.

Original code in game G_1

Decaps($c \neq c^*$):

$m' := \text{Dec}(\text{sk}, c)$

if $m' = \perp$

return $K := H(s, c)$

return $K := H(m', c)$

$H(m, c)$:

(Abort if BAD)

if $\exists K$ such that

$(m, c, K) \in T_H$

return K

$K \xleftarrow{\$} \{0,1\}^n$

$T_H := T_H \cup \{(m, c, K)\}$

Changed code in game G_2

Decaps($c \neq c^*$):

if $\exists K$ such that $(c, K) \in T_D$

return K

$K \xleftarrow{\$} \mathcal{K}$

$T_D := T_D \cup \{(c, K)\}$

return K

$H(m, c)$:

(Abort if BAD)

Lazy sample.

If $m \in M$: ensure

Decaps consistency

via $\text{PTcheck}(m, c)$

and table T_D

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle

Security claim (PKE OW-PCA $\Rightarrow$ KEM $^{\perp}$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $^{\perp}$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^{\perp}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions: G_2 to G_3

G_2 : Replace Decaps, H by to-be-specified simulations that work without oracle VAL.

G_3 : Abort if CHALL happens.

We build OW-PCA adversary $\mathcal{B}$ that simulates G_2 and wins if CHALL happens.

$$|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}).$$

Games G_2, G_3

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps($c \neq c^*$):

...

$H(m, c)$:

(Abort if BAD)

if $(m, c) = (m^*, c^*)$

and query was by $\mathcal{A}$

CHALL := true

Abort

Lazy sample.

If $m \in M$: ensure

Decaps consistency

via PTcheck(m, c)

and table T_D (like G_1

in the prior proof for

KEM $^{\perp}$)

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle

Security claim (PKE OW-PCA $\Rightarrow$ KEM $^{\perp}$ IND-C $\underline{\text{C}}$ A)

For any IND-C $\underline{\text{C}}$ A adversary $\mathcal{A}$ against KEM $^{\perp}$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^{\perp}}^{\text{IND-C}\underline{\text{C}}\text{A}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Game transitions: We collect the bounds:

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq q_H/|\mathcal{M}|.$$

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] = \Pr[G_2^{\mathcal{A}} \Rightarrow 1].$$

$$|\Pr[G_2^{\mathcal{A}} \Rightarrow 1] - \Pr[G_3^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}) \text{ and this} = 1/2$$

So

$$\text{Adv}_{\text{KEM}^{\perp}}^{\text{IND-C}\underline{\text{C}}\text{A}}(\mathcal{A}) \leq q_H/|\mathcal{M}| + \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}).$$

Games G_2, G_3

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps($c \neq c^*$):

...

$H(m, c)$:

(Abort if BAD)

if $(m, c) = (m^*, c^*)$

and query was by $\mathcal{A}$

CHALL := true

Abort

Lazy sample.

If $m \in M$: ensure

Decaps consistency

via PTcheck(m, c)

and table T_D (like G_1

in the prior proof for

KEM $^{\perp}$)

Fujisaki-Okamoto

Step 3: modify U-transform to avoid Valid oracle

Security claim (PKE OW-PCA $\Rightarrow$ KEM $^{\perp}$ IND-CCA)

For any IND-CCA adversary $\mathcal{A}$ against KEM $^{\perp}$, issuing at most q_H queries to H , there exists an OW-PCA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}^{\perp}}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(\mathcal{B}) + q_H/|\mathcal{M}|.$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Key insights:

We used:

- **Lazy sampling** and **Unpredictability** as before.
- Rejection of invalid ciphertexts with $K := H(s, c)$ instead of $\perp$ “hides” the rejection branch to some extent $\Rightarrow$ easier to simulate.
- **“Observable” ROM**: $\mathcal{B}$ found its one-way solution in $\mathcal{A}$ ’s RO queries.

Games G_2, G_3

Initialize:

...

Finalize ($b' \in \{0,1\}$):

...

Decaps($c \neq c^*$):

...

$H(m, c)$:

(Abort if BAD)

if $(m, c) = (m^*, c^*)$
and query was by $\mathcal{A}$

CHALL := true

Abort

Lazy sample.

If $m \in M$: ensure
Decaps consistency
via PTcheck(m, c)
and table T_D (like G_1
in the prior proof for
KEM $^{\perp}$)

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Now: Two ways to enable IND-CCA without requiring full OW-PCVA from PKE:

1. Augment the transform itself so that PKE's security requirement no longer involves **Valid**.
2. **Show that for deterministic PKEs, PKE's security requirement does not need to involve **PTcheck**.**

We do this for a cheaper FO variant, $\text{KEM}_m^\perp$, that does not need to hash c . We will assume OW in the presence of oracle **Valid**, which we call OW-VA.

Definition $\text{KEM}_m^\perp = U_m^\perp[\text{PKE}, H]$

<u>Encaps(pk):</u> $m \xleftarrow{\$} \mathcal{M}$ $c \leftarrow \text{Enc}(pk, m)$ $K := H(m)$ return (K, c)	<u>Decaps(sk, c):</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ else return $K := H(m')$
---	--

• Simulating this without sk needed **Valid**

• Simulating this without sk needed **PTcheck**.

Now: We don't need **PTcheck** if PKE is deterministic.

Game $\text{OWPCVA}_{\text{PKE}}^{\mathcal{A}}$

<u>Initialize:</u> $(pk, sk) \leftarrow \text{KeyGen}$ $m^* \xleftarrow{\$} \mathcal{M}$ $c^* \leftarrow \text{Enc}(pk, m^*)$ return pk, c^*	<u>PTcheck</u> $(m \in \mathcal{M}, c \in \mathcal{C})$ $m' := \text{Dec}(sk, c)$ return $m' =_? m$
<u>Finalize</u> $(m' \in \mathcal{M})$: return $m' =_? m^*$	<u>Valid</u> $(c \in \mathcal{C})$ $m' := \text{Dec}(sk, c)$ If $m' \in \mathcal{M}$ return true else return false

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (dPKE OW-VA $\Rightarrow$ $\text{KEM}_m^\perp$ IND-CCA)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

The proof somewhat resembles the one for $\text{KEM}^\perp$:

G_0 = IND-CPA game for $\text{KEM}_m^\perp$.

G_1 : Replace Decaps, H by simulations.

G_2 : Abort if H gets queried on m^*, c^* (CHALL).

Using these transitions, show that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}} \leq \Pr[\text{CHALL}] \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B})$$

Decaps and H in game $\text{IND-CCA}_{\text{KEM}^\perp}^{\mathcal{A}}$

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m, c)$</u> :
$m' := \text{Dec}(\text{sk}, c)$	if $\exists K$ such that $(m, c, K) \in T_H$
if $m' = \perp$	return K
return $\perp$	$K \xleftarrow{\$} \{0,1\}^n$
$K := H(m, c)$	$T_H := T_H \cup \{(m, c, K)\}$
return K	return K

Reminder: Simulations in $\text{KEM}^\perp$ proof

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m, c)$</u> :
if Valid (c) $\notin \mathcal{M}$	if $\exists K$ such that $(m, c, K) \in T_H$
return $\perp$	return K
if $\exists K$ such that	$K \xleftarrow{\$} \mathcal{K}$
$(c, K) \in T_D$	if PTcheck (m, c)
return K	if $\exists K'$ such that $(c, K') \in T_D$
$K \xleftarrow{\$} \mathcal{K}$	$K := K'$
$T_D := T_D$	else
$\cup \{(c, K)\}$	$T_D := T_D \cup \{(c, K)\}$
return K	$T_H := T_H \cup \{(m, c, K)\}$
	return K

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (dPKE OW-VA $\Rightarrow$ KEM $^\perp_m$ IND-CCA)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against KEM $^\perp_m$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

The proof somewhat resembles the one for KEM $^\perp$:

G_0 = IND-CPA game for KEM $^\perp_m$.

G_1 : Replace Decaps, H by simulations.

G_2 : Abort if H gets queried on m^*, c^* (CHALL).

Now: Change the proof - we replace the simulation of H with one that no longer hashes c and no longer uses **PTcheck**.

Decaps and H in game IND-CCA $^\mathcal{A}_{\text{KEM}^\perp}$

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m, c)$</u> :
$m' := \text{Dec}(\text{sk}, c)$	if $\exists K$ such that $(m, c, K) \in T_H$
if $m' = \perp$	return K
return $\perp$	$K \xleftarrow{\$} \{0,1\}^n$
$K := H(m, c)$	$T_H := T_H \cup \{(m, c, K)\}$
return K	return K

Reminder: Simulations in KEM $^\perp$ proof

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m, c)$</u> :
if Valid (c) $\notin \mathcal{M}$	if $\exists K$ such that $(m, c, K) \in T_H$
return $\perp$	return K
if $\exists K$ such that	$K \xleftarrow{\$} \mathcal{K}$
$(c, K) \in T_D$	if PTcheck (m, c)
return K	if $\exists K'$ such that $(c, K') \in T_D$
$K \xleftarrow{\$} \mathcal{K}$	$K := K'$
$T_D := T_D$	else
$\cup \{(c, K)\}$	$T_D := T_D \cup \{(c, K)\}$
return K	$T_H := T_H \cup \{(m, c, K)\}$
	return K

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (**dPKE OW-VA** $\Rightarrow$ $\text{KEM}_m^\perp$ **IND-CCA**)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

The proof somewhat resembles the one for $\text{KEM}^\perp$:

G_0 = IND-CPA game for $\text{KEM}_m^\perp$.

G_1 : Replace Decaps, H by simulations.

G_2 : Abort if H gets queried on m^*, c^* (CHALL).

Now: Change the proof - we replace the simulation of H with one that no longer hashes c and no longer uses **PTcheck**.

H 's simulation in $\text{KEM}^\perp$ proof

$H(m, c)$:

...

if **PTcheck** (m, c)

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

New simulation of H

$H(m)$:

...

$c := \text{Enc}(\text{pk}, m)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (**dPKE OW-VA** $\Rightarrow$ $\text{KEM}_m^\perp$ **IND-CCA**)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

The proof somewhat resembles the one for $\text{KEM}^\perp$:

G_0 = IND-CPA game for $\text{KEM}_m^\perp$.

G_1 : Replace Decaps, H by simulations.

G_2 : Abort if H gets queried on m^*, c^* (CHALL).

Now: Show that this does not really change the proof.

H 's simulation in $\text{KEM}^\perp$ proof

$H(m, c)$:

...

if **PTcheck** (m, c)

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

New simulation of H

$H(m)$:

...

$c := \text{Enc}(\text{pk}, m)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

KEM[⊥] proof: Using **PTcheck** and table management ensured consistency, i.e. that $\text{Decaps}(c) = H(\text{Dec}(\text{sk}, c), c)$.

Now:

Show we ensure the same thing by setting $c := \text{Enc}(\text{pk}, m)$.

All we need to show is:

$\text{PTcheck}(m, c)$ is true if and only if $\text{Enc}(\text{pk}, m) = c$.

In other words,

$\text{Dec}(\text{sk}, c) = m$ if and only if $\text{Enc}(\text{pk}, m) = c$.

This is true due to correctness: If $\text{Enc}(\text{pk}, m) = c$, then

$$\text{Dec}(\text{sk}, c) = \text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m.$$

And the other way around, if $\text{Dec}(\text{sk}, c) = m$, then

$$\text{Enc}(\text{pk}, m) = \text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) = c.$$

H's simulation in KEM[⊥] proof

H(m, c):

...

if **PTcheck(m, c)**

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

New simulation of H

H(m):

...

$c := \text{Enc}(\text{pk}, m)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (dPKE OW-VA $\Rightarrow$ KEM $^\perp_m$ IND-CCA)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against KEM $^\perp_m$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$Adv_{KEM^\perp_m}^{IND-CCA}(\mathcal{A}) \leq Adv_{PKE}^{OW-VA}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Our new proof for KEM $^\perp_m$:

G_0 = IND-CPA game for KEM $^\perp_m$.

G_1 : Replace Decaps, H by simulations.

G_2 : Abort if H gets queried on m^*, c^* (CHALL).

Since the simulations work, we bound (like before)

$$Adv_{KEM^\perp_m}^{IND-CCA}(\mathcal{A}) \leq \Pr[\text{CHALL}].$$

Decaps and H in game IND-CCA $^\mathcal{A}_{KEM^\perp_m}$

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
$m' := \text{Dec}(\text{sk}, c)$	if $\exists K$ such that $(m, K) \in T_H$
if $m' = \perp$	return K
return $\perp$	$K \xleftarrow{\$} \{0,1\}^n$
$K := H(m)$	$T_H := T_H \cup \{(m, K)\}$
return K	return K

New simulations

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
if Valid (c) $\notin \mathcal{M}$	if $\exists K$ such that $(m, K) \in T_H$
return $\perp$	return K
if $\exists K$ such that	$K \xleftarrow{\$} \mathcal{K}$
$(c, K) \in T_D$	$c := \text{Enc}(\text{pk}, m)$
return K	if $\exists K'$ such that $(c, K') \in T_D$
$K \xleftarrow{\$} \mathcal{K}$	$K := K'$
$T_D := T_D$	else
$\cup \{(c, K)\}$	$T_D := T_D \cup \{(c, K)\}$
return K	$T_H := T_H \cup \{(m, K)\}$
	return K

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (dPKE OW-VA $\Rightarrow$ KEM $^\perp_m$ IND-CCA)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against KEM $^\perp_m$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

It remains to build OW-VA adversary $\mathcal{B}$ and show

$$\Pr[\text{CHALL}] \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}).$$

To solve its OW challenge, $\mathcal{B}$

- goes through the hash table T_H ,
- finds the tuple (m, K) such that $\text{Enc}(\text{pk}, m) = c^*$ (which exists if CHALL happened),
- and outputs m as its solution.

Decaps and H in game IND-CCA $^\mathcal{A}_{\text{KEM}_m^\perp}$

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
$m' := \text{Dec}(\text{sk}, c)$	if $\exists K$ such that $(m, K) \in T_H$
if $m' = \perp$	return K
return $\perp$	$K \xleftarrow{\$} \{0,1\}^n$
$K := H(m)$	$T_H := T_H \cup \{(m, K)\}$
return K	return K

New simulations

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
if <u>Valid</u> (c) $\notin \mathcal{M}$	if $\exists K$ such that $(m, K) \in T_H$
return $\perp$	return K
if $\exists K$ such that	$K \xleftarrow{\$} \mathcal{K}$
$(c, K) \in T_D$	$c := \text{Enc}(\text{pk}, m)$
return K	if $\exists K'$ such that $(c, K') \in T_D$
$K \xleftarrow{\$} \mathcal{K}$	$K := K'$
$T_D := T_D$	else
$\cup \{(c, K)\}$	$T_D := T_D \cup \{(c, K)\}$
return K	$T_H := T_H \cup \{(m, K)\}$
	return K

Fujisaki-Okamoto

Step 4: dPKEs avoid Ptcheck requirement

Security claim (dPKE OW-VA $\Rightarrow$ KEM $^\perp_m$ IND-CCA)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against KEM $^\perp_m$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

It remains to build OW-VA adversary $\mathcal{B}$ and show

$$\Pr[\text{CHALL}] \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}).$$

To solve its OW challenge, $\mathcal{B}$

- goes through the hash table T_H ,
- finds the tuple (m, K) such that $\text{Enc}(\text{pk}, m) = c^*$ (which exists if CHALL happened),
- and outputs m as its solution.

Decaps and H in game IND-CCA $^\mathcal{A}_{\text{KEM}_m^\perp}$

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
$m' := \text{Dec}(\text{sk}, c)$	if $\exists K$ such that $(m, K) \in T_H$
if $m' = \perp$	return K
return $\perp$	$K \xleftarrow{\$} \{0,1\}^n$
$K := H(m)$	$T_H := T_H \cup \{(m, K)\}$
return K	return K

New simulations

<u>Decaps</u> ($c \neq c^*$):	<u>$H(m)$</u> :
if Valid (c) $\notin \mathcal{M}$	if $\exists K$ such that $(m, K) \in T_H$
return $\perp$	return K
if $\exists K$ such that	$K \xleftarrow{\$} \mathcal{K}$
$(c, K) \in T_D$	$c := \text{Enc}(\text{pk}, m)$
return K	if $\exists K'$ such that $(c, K') \in T_D$
$K \xleftarrow{\$} \mathcal{K}$	$K := K'$
$T_D := T_D$	else
$\cup \{(c, K)\}$	$T_D := T_D \cup \{(c, K)\}$
return K	$T_H := T_H \cup \{(m, K)\}$
	return K

Waiiiit a minute.

Security claim (**dPKE OW-VA** $\Rightarrow$ $\text{KEM}_m^\perp$ **IND-CCA**)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Waiiiit a minute.

Security claim ($\text{dPKE OW-VA} \Rightarrow \text{KEM}_m^\perp \text{ IND-CCA}$)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Look at the following PKE scheme!

Let PKE be a OW-VA secure encryption scheme with ciphertext space $\mathcal{C}$. We build a new PKE':

- $\mathcal{C}' = \{0,1\} \times \mathcal{C}$,
- $\text{Enc}'(pk, m) = 0 \parallel \text{Enc}(pk, m)$,
- $\text{Dec}'(sk, b \parallel c) = \text{Dec}(sk, c)$.

I claim: If PKE is OW-VA, then so is PKE'.

And yet: If we build $\text{KEM}_m^\perp$ from PKE', **the result is not IND-CCA secure!**

Waiiiit a minute.

If PKE is OW-VA, then so is PKE'

Game $\text{OWVA}_{\text{PKE}}^{\mathcal{B}}$

Initialize:
 $(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $c^* \leftarrow \text{Enc}(pk, m^*)$
return pk, c^*

Valid($c \in \mathcal{C}$)
 $m' := \text{Dec}(sk, c)$
If $m' \in \mathcal{M}$ return true
else return false

Finalize($m' \in \mathcal{M}$):
return $m' =_? m^*$

Game $\text{OWVA}_{\text{PKE}'}^{\mathcal{A}}$

Initialize:
 $(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $c^* \leftarrow \text{Enc}(pk, m^*)$
return $pk, 0 || c^*$

Valid($b || c$)
 $m' := \text{Dec}(sk, c)$
If $m' \in \mathcal{M}$ return true
else return false

Finalize($m' \in \mathcal{M}$):
return $m' =_? m^*$

We show that for any OW-VA adversary $\mathcal{A}$, we can build an OW-VA adversary $\mathcal{B}$ s.th.

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}) :$$

$\mathcal{B}$ can easily simulate game $\text{OWVA}_{\text{PKE}'}$ to $\mathcal{A}$:

$\mathcal{B}$ calls Initialize, gets pk, c^* , and responds to $\mathcal{A}$'s Initialize call with $pk, 0 || c^*$.

$\mathcal{B}$ simulates Valid by dropping b and forwarding the call to its own Valid oracle.

To win the OW challenge, $\mathcal{B}$ forwards $\mathcal{A}$'s call to Finalize to its own Finalize.

Waiiiit a minute.

The resulting KEM is NOT IND-CCA secure

Question

Do you see an attack?

$\mathcal{A}$ gets the challenge pk, c^*, K_b^* with $c^* = 0||c'$

$\mathcal{A}$ mals the ciphertext by flipping the first bit in c^* :

$$c_{BAD} := 1||c'.$$

No one forbids $\mathcal{A}$ to call Decaps on c_{BAD} since $c_{BAD} \neq c^*$.

Decaps computes:

$$m' := \text{Dec}(sk, c') = \text{Dec}(sk, \text{Enc}(pk, m^*)) = m^*$$

and outputs

$$H(m^*) = K_0^* !!$$

Games $\text{IND-CCA}_{\text{KEM}_m^\perp}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$b \xleftarrow{\$} \{0,1\}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c' := \text{Enc}(pk, m^*)$

$c^* := 0||c'$

$K_0^* := H(m^*)$

$K_1^* \xleftarrow{\$} \mathcal{K}$

return pk, c^*, K_b^*

Decaps($(b||c) \neq c^*$):

$m' := \text{Dec}(sk, c)$

if $m' = \perp$

return $\perp$

else

return $K := H(m')$

Finalize ($b' \in \{0,1\}$):

return $b' =_? b$

Waiiiit a minute.

Security claim (**dPKE OW-VA** $\Rightarrow$ $\text{KEM}_m^\perp$ **IND-CCA**)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

So we found a PKE scheme that is OW-VA and yet $\text{KEM}_m^\perp$ **is not IND-CCA secure**?!

Question

How the hell can that be?

Waiiiit a minute.

Security claim (**dPKE OW-VA** $\Rightarrow$ $\text{KEM}_m^\perp$ **IND-CCA**)

Let PKE be *deterministic*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

So we found a PKE scheme that is OW-VA and yet $\text{KEM}_m^\perp$ **is not IND-CCA secure**?!

Question

Did we mess up the proof?

Fujisaki-Okamoto

Step 4: dPKEs, fixing the proof

We wanted to show: we still ensure consistency, i.e.
 $\text{Decaps}(c) = H(\text{Dec}(\text{sk}, c))$.

So we needed to show:

$\text{Dec}(\text{sk}, c) = m$ if and only if $\text{Enc}(\text{pk}, m) = c$.

We argued: if $\text{Dec}(\text{sk}, c) = m$, then

$\text{Enc}(\text{pk}, m) = \text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) = c$.

But **this** is not necessarily true! There could exist two distinct ciphertexts $c_1 \neq c_2$ that decrypt to the same message m ! Then,

- Decaps would answer both queries with $H(\text{Dec}(\text{sk}, c))$, but
- Our simulated Decaps would not! Say m encrypts to c_1 . For c_2 , or simulation will miss the connection!

We handle this by introducing a new PKE property that makes this case impossible, called **rigidity**.

H 's simulation in $\text{KEM}^\perp$ proof

$H(m, c)$:

...

if $\text{PTcheck}(m, c)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

New simulation of H

$H(m)$:

...

$c := \text{Enc}(\text{pk}, m)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

Fujisaki-Okamoto

Step 4: dPKEs, fixing the proof

Definition **Rigidity**

We call a deterministic public-key encryption scheme PKE ***rigid*** if for all $(pk, sk) \leftarrow \text{KeyGen}$ and all ciphertexts c , we have that

$$\text{Dec}(sk, c) = \perp \text{ or } \text{Enc}(pk, \text{Dec}(sk, c)) = c .$$

Fujisaki-Okamoto

Step 4: dPKEs, fixing the proof

We wanted to show: we still ensure consistency, i.e.
 $\text{Decaps}(c) = H(\text{Dec}(\text{sk}, c))$.

In the proof, we had the issue that even if $\text{Dec}(\text{sk}, c) = m$, we could not rule out that

$$\text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) \neq c ,$$

and **this case broke our simulation**.

If PKE is rigid, this case is actually ruled out:

If PKE is rigid, then $\text{Dec}(\text{sk}, c) = \perp$ or $\text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) = c$.

Since we are in the case that $\text{Dec}(\text{sk}, c) = m \neq \perp$, the second option now **must** hold:

$$\text{Enc}(\text{pk}, \text{Dec}(\text{sk}, c)) = c .$$

This fixes our proof. (-_-")

H's simulation in $\text{KEM}^\perp$ proof

$H(m, c)$:

...

if $\text{PTcheck}(m, c)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

New simulation of H

$H(m)$:

...

$c := \text{Enc}(\text{pk}, m)$

if $\exists K'$ such that $(c, K') \in T_D$

$K := K'$

else

$T_D := T_D \cup \{(c, K)\}$

...

Fujisaki-Okamoto

Step 4: dPKEs, fixing the proof

We correct our security claim:

Security claim (dPKE OW-VA $\Rightarrow$ $\text{KEM}_m^\perp$ IND-CCA)

Let PKE be deterministic *and rigid*. For any IND-CCA adversary $\mathcal{A}$ against $\text{KEM}_m^\perp$, there exists a OW-VA adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{KEM}_m^\perp}^{\text{IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-VA}}(\mathcal{B}).$$

$\mathcal{B}$ runs in about the same time as $\mathcal{A}$.

Question

What about the counterexample?

Fujisaki-Okamoto

Step 4: dPKEs, fixing the proof

Rigidity also rules out counterexample PKE' as PKE' is not rigid:

- $\text{Enc}'(pk, m) = 0 \parallel \text{Enc}(pk, m)$,
- $\text{Dec}'(sk, b \parallel c') = \text{Dec}(sk, c')$.

For rigidity, we would need to have that

$$\text{Dec}'(sk, c) = \perp \text{ or that } \text{Enc}'(\text{Dec}'(sk, c)) = c.$$

Let $0 \parallel \text{Enc}(pk, m)$ be the encryption of any message m .

Set $c := 1 \parallel \text{Enc}(pk, m)$.

We check:

$$\text{Dec}'(sk, c) = \text{Dec}(sk, \text{Enc}(pk, m)) = m \neq \perp ,$$

but also

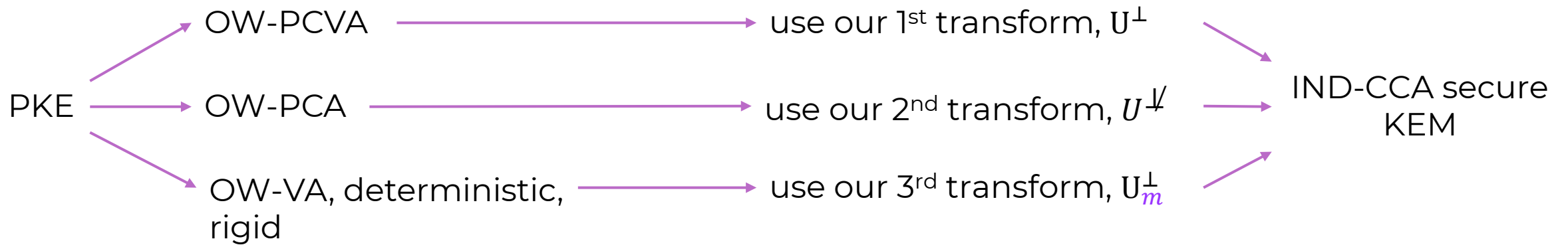
$$\text{Enc}'(\text{Dec}'(sk, c)) = \text{Enc}'(m) = 0 \parallel \text{Enc}(pk, m) \neq c .$$

So c violates the rigidity condition.

Fujisaki-Okamoto

What did we learn about U-type transforms?

We can (tightly) build an IND-CCA secure KEM from PKE as follows:



So far, we did not enable results for probabilistic PKEs that are “plain” OW. ☹️

	Key definition $K := H(m, c)$	$K := H(m)$
Handling of invalid cts		
Respond with $\perp$	$U^\perp$	$U_m^\perp$
Respond with $H(s, c)$, using secret s („implicit rejection“)	$U^{\not\perp}$	$U_m^{\not\perp}$ (not covered)

We could (but won't) combine the proofs for $U_m^\perp$, $U^{\not\perp}$ into one for $U_m^{\not\perp}$, requiring only a rigid dPKE that is “plain” OW.

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We define a transform that takes a PKE scheme with a probabilistic encryption algorithm and derandomizes that one.

Definition T-transform

Let $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ have message space $\mathcal{M}$ and encryption randomness space $\mathcal{R}$. Let $G: \mathcal{M} \rightarrow \mathcal{R}$ be a hash function. We define PKE with KeyGen and algorithms

<u>$\text{Enc}'(pk, m)$:</u> $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	<u>$\text{Dec}'(sk, c)$:</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Use r as
Enc's coin

"Re-encryption check"
notices "improper" c

This PKE-to-dPKE transform is called the **T transform**, and the resulting PKE is denoted as TPKE or $T[\text{PKE}, G]$.

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

- rigid;
- plain OW;
- OW-PCA, by showing how to simulate PTcheck,
- OW-VA, by showing how to simulate Valid,
- thus also OW-PCVA (since both oracles are simulatable).

$\text{Enc}'(pk, m):$ $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	$\text{Dec}'(sk, c):$ $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE
TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

- **rigid**;
- plain OW;
- OW-PCA, by showing how to simulate PTcheck,
- OW-VA, by showing how to simulate Valid,
- thus also OW-PCVA (since both oracles are simulatable).

$\text{Enc}'(pk, m):$ $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	$\text{Dec}'(sk, c):$ $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We check that TPKE is rigid:

We need to check: for all $(pk, sk) \leftarrow \text{KeyGen}$ and all c :

If

$$m' := \text{Dec}'(sk, c) \neq \perp,$$

then

$$\text{Enc}'(pk, m') = c.$$

So fix any (pk, sk) , any c , and assume $m' := \text{Dec}'(sk, c) \neq \perp$. We verify that then, $\text{Enc}'(pk, m') = c$:

Dec' computes $m' := \text{Dec}(sk, c)$, returning $\perp$ if $m' = \perp$.
 Dec' then “re-encryption checks”: returns $\perp$ unless it holds that $\text{Enc}'(pk, m') = c$.

So for $\text{Dec}'(sk, c) \neq \perp$ to hold, c must pass this check and thereby satisfy the rigidity condition.

<u>$\text{Enc}'(pk, m)$:</u> $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	<u>$\text{Dec}'(sk, c)$:</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

- ✓ rigid;
- **plain OW**;
- OW-PCA, by showing how to simulate PTcheck,
- OW-VA, by showing how to simulate Valid,
- thus also OW-PCVA (since both oracles are simulatable).

$\text{Enc}'(pk, m):$ $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	$\text{Dec}'(sk, c):$ $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exists a OW adversary $\mathcal{B}$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}).$$

Proof intuition:

$$\text{OW}_{\text{PKE}}^{\mathcal{A}} = \text{OW}_{\text{PKE}'}^{\mathcal{A}}$$

$\mathcal{B}$ just forwards $\mathcal{A}$'s response.

Question

... right?

Game $\text{OW}_{\text{PKE}}^{\mathcal{B}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that $(m, r) \in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Game $\text{OW}_{\text{PKE}'}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$r^* := G(m)$

$c^* \leftarrow \text{Enc}(pk, m^*; r^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that $(m, r) \in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

~~Security claim (PKE OW $\Rightarrow$ PKE' OW)~~

~~For any OW adversary $\mathcal{A}$ against PKE, issuing at most q_G queries to random oracle G , there exists a OW adversary $\mathcal{B}$ against PKE such that $\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B})$.~~

Proof intuition:

$$\text{OW}_{\text{PKE}}^{\mathcal{A}} = \text{OW}_{\text{PKE}'}^{\mathcal{A}}$$

$\mathcal{B}$ just forwards $\mathcal{A}$'s response.

Question

... right?

Game $\text{OW}_{\text{PKE}}^{\mathcal{B}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that (m, r)

$\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Game $\text{OW}_{\text{PKE}'}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$r^* := G(m)$

$c^* \leftarrow \text{Enc}(pk, m^*; r^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that (m, r)

$\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

$\text{OW}_{\text{PKE}}^{\mathcal{A}} = \text{OW}_{\text{PKE}'}^{\mathcal{A}}$ **unless $\mathcal{A}$ queries G on m^* .**

Case 1 = no such query: $\mathcal{A}$ exactly breaks PKE. So does $\mathcal{B}_1$ by forwarding m' .

Case 2 = **QUERY** happens: implies knew m^* , so $\mathcal{A}$ also broke PKE. Which $\mathcal{B}_2$ exploits by monitoring G .

Game $\text{OW}_{\text{PKE}}^{\mathcal{B}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$c^* \leftarrow \text{Enc}(pk, m^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that $(m, r) \in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Game $\text{OW}_{\text{PKE}'}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$r^* := G(m)$

$c^* \leftarrow \text{Enc}(pk, m^*; r^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that $(m, r) \in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

$\text{OW}_{\text{PKE}}^{\mathcal{A}} = \text{OW}_{\text{PKE}'}^{\mathcal{A}}$ **unless $\mathcal{A}$ queries G on m^* .**

Case 1 = no such query: $\mathcal{A}$ exactly breaks PKE. So does $\mathcal{B}_1$ by forwarding m' .

Case 2 = **QUERY** happens: implies knew m^* , so $\mathcal{A}$ also broke PKE. Which $\mathcal{B}_2$ exploits by monitoring G .

Games G_0, G_1

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $m = m^*$

And query by $\mathcal{A}$

QUERY := true

Abort

if $\exists r$ such that (m, r)
 $\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Game transitions: G_0 to G_1

G_0 = OW game for PKE'.

G_1 : Abort upon QUERY, so if G gets queried on m^* .

Since G_1 = OW game for PKE (unless QUERY happens), for the „just-forward“ reduction $\mathcal{B}_1$ it holds that

$$\Pr[G_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1).$$

Games G_0, G_1

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $m = m^*$

And query by $\mathcal{A}$

QUERY := true

Abort

if $\exists r$ such that (m, r)

$\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Game transitions: G_0 to G_1

G_0 = OW game for PKE'.

G_1 : Abort upon QUERY, so if G gets queried on m^* .

$$|\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\text{QUERY}].$$

Games G_0, G_1

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $m = m^*$

And query by $\mathcal{A}$

QUERY := true

Abort

if $\exists r$ such that (m, r)

$\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Game transitions: G_0 to G_1

G_0 = OW game for PKE'.

G_1 : Abort upon QUERY, so if G gets queried on m^* .

We bound $\Pr[\text{QUERY}]$: OW adversary $\mathcal{B}_2$ simulates G_0 , randomly picking a query to G as its guess for m^* .

$$\Pr[\text{QUERY}] \leq q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Games G_0, G_1

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $m = m^*$

And query by $\mathcal{A}$

QUERY := true

Abort

if $\exists r$ such that (m, r)

$\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We first show: the derandomized PKE inherits OW.

Security claim (PKE OW $\Rightarrow$ PKE' OW)

For any OW adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Key insights:

We used:

- **Lazy sampling** and **Unpredictability** as before.
- We need to be careful about when RO values are being used in challenges.
- **“Observable” ROM**: $\mathcal{B}_2$ found its one-way solution in $\mathcal{A}$'s RO queries.

Games G_0, G_1

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $m = m^*$

And query by $\mathcal{A}$

QUERY := true

Abort

if $\exists r$ such that (m, r)
 $\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$
return r

Finalize ($m' \in \mathcal{M}$):
return $m' =_? m^*$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

- ✓ rigid;
- ✓ plain OW;
- **OW-PCA**, by showing how to simulate **PTcheck**,
- OW-VA, by showing how to simulate **Valid**,
- thus also OW-PCVA (since both oracles are simulatable).

<u>$\text{Enc}'(pk, m):$</u> $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	<u>$\text{Dec}'(sk, c):$</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

We already showed the OW part.

So we only need to show that $\mathcal{B}_1$ and $\mathcal{B}_2$ also could
simulate oracle PTcheck.

Simulation idea: If a c decrypts to a plaintext m ,
then m encrypts to c . This we can check without sk .

Game $\text{OWPCA}_{\text{PKE}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$

$m^* \xleftarrow{\$} \mathcal{M}$

$r^* := G(m)$

$c^* \leftarrow \text{Enc}(pk, m^*; r^*)$

return pk, c^*

$G(m)$

if $\exists r$ such that (m, r)
 $\in T_G$

return r

$r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$

return r

Finalize ($m' \in \mathcal{M}$):

return $m' =_? m^*$

PTcheck ($m \in \mathcal{M}, c \in \mathcal{C}$)

// $m' := \text{Dec}'(sk, c)$:

$m' := \text{Dec}(sk, c)$

if $m' = \perp$

return false

$r' := G(m')$

$c' := \text{Enc}(pk, m'; r')$

if $c' \neq c$ return false

return $m' =_? m$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

We already showed the OW part.

So we only need to show that $\mathcal{B}_1$ and $\mathcal{B}_2$ also could
simulate oracle PTcheck.

Simulation idea: If a c decrypts to a plaintext m ,
then m encrypts to c . This we can check without sk .

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(sk, c)$  :  
   $m' := \text{Dec}(sk, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(pk, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(pk, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Question

Is this simulation
convincing?

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We check that the code does exactly the same:

Consider a single query (m, c) , set $m' := \text{Dec}(\text{sk}, c)$.

- If PTcheck says “true”, so does our simulation:

$$\text{PTcheck}(m, c) = \text{true} \Rightarrow m' = m \text{ and } \text{Enc}(\text{pk}, m'; G(m')) = c ,$$

so our simulation also says “true”.

- If our simulation says “true”, so does PTcheck:

If our simulation says true, then $\text{Enc}(\text{pk}, m; G(m)) = c$.

Due to correctness of PKE,

$$m' = m$$

so PTcheck also says “true”.

So both oracles always give the same response.

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Question

What about the
loss, q_G ?

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

$\mathcal{B}_2$ wins if QUERY happens, by randomly picking a
query to G as its guess for m^* . We concluded

$$\Pr[\text{QUERY}] \leq q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

But our simulated PTcheck also calls G once per
execution!

So G 's table size will be $q_G + \# \text{ PTcheck calls}$.

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-PCA.

~~Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)~~

~~For any OW-PCA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that~~

$$\text{Adv}_{\text{PKE}' \text{ OW-PCA}}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2)$$

$\mathcal{B}_2$ wins if QUERY happens, by randomly picking a query to G as its guess for m^* . We concluded

$$\Pr[\text{QUERY}] \leq q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

But our simulated PTcheck also calls G once per execution!

So G 's table size will be $q_G + \# \text{ PTcheck calls}$.

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We correct the statement:

Security claim (PKE OW $\Rightarrow$ PKE' OW-PCA)

For any OW-PCA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G **and q_P many to oracle PTcheck**, there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-PCA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + (q_G + q_P) \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Original oracle

```
PTcheck( $m \in M, c \in C$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  return  $m' =_? m$ 
```

Simulation

```
PTcheck( $m \in M, c \in C$ )  
 $r := G(m)$   
 $c' := \text{Enc}(\text{pk}, m; r)$   
return  $c' =_? c$ 
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

- ✓ rigid;
- ✓ plain OW;
- ✓ OW-PCA, by showing how to simulate PTcheck,
- **OW-VA, by showing how to simulate Valid,**
- ✓ **thus also OW-PCVA (since both oracles are simulatable).**

<u>$\text{Enc}'(pk, m):$</u> $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	<u>$\text{Dec}'(sk, c):$</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-VA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

We already showed the OW part. We only need to
show how $\mathcal{B}_1$ and $\mathcal{B}_2$ can simulate oracle **Valid**.

Simulation idea: c is built in a very specific way, as
 $\text{Enc}(\text{pk}, m; G(m))$. Without calling $G(m)$, $\mathcal{A}$ can't build
any c that **passes re-encryption**.

$\mathcal{B}$ will recognize all valid c by monitoring T_G .

Game $\text{OWVA}_{\text{PKE}}^{\mathcal{A}}$

Initialize:

$(pk, sk) \leftarrow \text{KeyGen}$
 $m^* \xleftarrow{\$} \mathcal{M}$
 $r^* := G(m)$
 $c^* \leftarrow \text{Enc}(pk, m^*; r^*)$
return pk, c^*

$G(m)$

if $\exists r$ such that (m, r)
 $\in T_G$
return r
 $r \xleftarrow{\$} \mathcal{R}$

$T_G := T_G \cup \{(m, r)\}$
return r

Finalize ($m' \in \mathcal{M}$):
return $m' =_? m^*$

Valid ($c \in \mathcal{C}$)

// $m' := \text{Dec}'(sk, c)$:
 $m' := \text{Dec}(sk, c)$
if $m' = \perp$
return false
 $r' := G(m')$
 $c' := \text{Enc}(pk, m'; r')$
if $c' \neq c$ return false
If $m' \in \mathcal{M}$ return true
else return false

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-VA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Proof intuition:

We already showed the OW part. We only need to
show how $\mathcal{B}_1$ and $\mathcal{B}_2$ can simulate oracle **Valid**.

Simulation idea: c is built in a very specific way, as
 $\text{Enc}(\text{pk}, m; G(m))$. Without calling $G(m)$, $\mathcal{A}$ can't build
any c that passes re-encryption.

$\mathcal{B}$ will recognize all valid c by monitoring T_G .

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We now show: we also achieve OW-VA.

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE',
issuing at most q_G queries to random oracle G ,
there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against
PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2).$$

Question

Is this simulation
convincing?

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We check that the code does **almost** the same:

Consider a single query (m, c) , set $m' := \text{Dec}(\text{sk}, c)$.

- If Valid says “false”, so does our simulation:

$$\text{VALID}(c) = \text{false} \Rightarrow m' = \perp \text{ or } \text{Enc}(\text{pk}, m'; G(m')) \neq c,$$

so our simulation also says “false”.

- If Valid says “true”, so does our simulation, **very likely**:

If VALID answers the query with true, so would our first helper simulation, **unless $G(m')$ was not queried yet**.

Then $\mathcal{A}$ managed to guess a ciphertext $c = \text{Enc}(\text{pk}, m; G(m))$ without knowing $G(m)$! Let's call this event **GUESS**.

We showed: the simulation is perfect up to event **GUESS**, which is **unlikely if ciphertexts are sufficiently hard to predict**.

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We formalize that ciphertexts should be “hard to predict”, by formalizing that they are “spread randomly enough”.

Definition γ -spreadness

We call a PKE γ -spread if, for every key pair $(pk, sk) \leftarrow \text{KeyGen}$ and every message $m \in \mathcal{M}$ and every possible ciphertext $c \in \mathcal{C}$,

$$\Pr_{r \leftarrow \mathcal{R}} [c = \text{Enc}(pk, m; r)] \leq 2^{-\gamma}.$$

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We check that the code does **almost** the same:

Consider a single query (m, c) , set $m' := \text{Dec}(\text{sk}, c)$.

- If Valid says “true”, so does our simulation, **very likely**:

If VALID answers the query with true, so would our first helper simulation, **unless $G(m')$ was not queried yet**.

Then $\mathcal{A}$ managed to guess a ciphertext $c = \text{Enc}(\text{pk}, m; G(m))$ without knowing $G(m)$! Let's call this event **GUESS**.

If PKE is γ -spread, then **GUESS** can only happen with probability at most $2^{-\gamma}$ per query to VALID.

The distinguishing advantage between Valid and our simulation is thus at most

$$q_V \cdot 2^{-\gamma} ,$$

where q_V is the number of queries to oracle VALID.

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We account for the simulation distinguishing advantage:

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2) + q_V \cdot 2^{-\gamma}.$$

Question

Can $\mathcal{B}_1, \mathcal{B}_2$ really simulate this?

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We account for the simulation distinguishing advantage:

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2) + q_V \cdot 2^{-\gamma}.$$

Simulation 1 still uses sk to find c 's "preimage" m' .

Idea: Instead of finding it via Dec, we just look it up T_G .

Original oracle

```
Valid( $c \in \mathcal{C}$ )  
//  $m' := \text{Dec}'(\text{sk}, c)$  :  
   $m' := \text{Dec}(\text{sk}, c)$   
  if  $m' = \perp$   
    return false  
   $r' := G(m')$   
   $c' := \text{Enc}(\text{pk}, m'; r')$   
  if  $c' \neq c$  return false  
  if  $m' \in \mathcal{M}$  return true  
  else return false
```

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
  return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
  return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We account for the simulation distinguishing advantage:

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2) + q_V \cdot 2^{-\gamma}.$$

Simulation 1 still uses sk to find c 's "preimage" m' .

Idea: Instead of finding it via Dec, we look it up in T_G .

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
    return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Simulation 2

```
Valid( $c \in \mathcal{C}$ )  
 $m^t := \text{Dec}(\text{sk}, c)$   
if  $m^t = \perp$   
    return false  
if  $\exists (m, r) \in T_G$  s. th.  
 $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We account for the simulation distinguishing advantage:

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2) + q_V \cdot 2^{-\gamma}.$$

Simulation 1 still uses sk to find c 's "preimage" m' .

Idea: Instead of finding it via Dec, we look it up in T_G .

Question

Is this simulation convincing?

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
    return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Simulation 2

```
Valid( $c \in \mathcal{C}$ )  
 $m^{\dagger} := \text{Dec}(\text{sk}, c)$   
if  $m^{\dagger} = \perp$   
    return false  
if  $\exists (m, r) \in T_G$  s. th.  
 $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We check that the code does exactly the same:

Consider a single query (m, c) , set $m' := \text{Dec}(\text{sk}, c)$.

- If Simulation 1 says “true”, so does simulation 2:

If Sim. 1 says “true”, then $(m', r) \in T_G$ and $\text{Enc}(\text{pk}, m'; r) = c$.

But then Sim. 2 finds (m', r) in T_G and also says “true”.

- If Simulation 2 says “true”, so does simulation 1:

If Sim. 2 says “true”, then there is **some** $(m, r) \in T_G$ s.th.

$\text{Enc}(\text{pk}, m; r) = c$.

By correctness, we have $m' = m$, so Sim. 1 also finds the tuple and says “true”:

$$m' := \text{Dec}(\text{sk}, c) = \text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m; r)) = m.$$

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
    return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Simulation 2

```
Valid( $c \in \mathcal{C}$ )  
 $m^t := \text{Dec}(\text{sk}, c)$   
if  $m^t = \perp$   
    return false  
if  $\exists (m, r) \in T_G$  s.th.  
 $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

Since the simulation works, we can conclude:

Security claim (PKE OW $\Rightarrow$ PKE' OW-VA)

For any OW-VA adversary $\mathcal{A}$ against PKE', issuing at most q_G queries to random oracle G , there exist two OW adversaries $\mathcal{B}_1, \mathcal{B}_2$ against PKE such that

$$\text{Adv}_{\text{PKE}'}^{\text{OW-VA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_1) + q_G \cdot \text{Adv}_{\text{PKE}}^{\text{OW}}(\mathcal{B}_2) + q_V \cdot 2^{-\gamma}.$$

Key insights:

We used:

- **Lazy sampling** and **Unpredictability** as before.
- **“Observable” ROM**: Not only does $\mathcal{B}_2$ find its one-way solution in $\mathcal{A}$'s RO queries. Observing the queries **also helps the simulation to pretend decrypting**.

Simulation 1

```
Valid( $c \in \mathcal{C}$ )  
 $m' := \text{Dec}(\text{sk}, c)$   
if  $m' = \perp$   
    return false  
if  $\exists r$  s. th.  $(m', r) \in T_G$   
and  $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Simulation 2

```
Valid( $c \in \mathcal{C}$ )  
 $m^t := \text{Dec}(\text{sk}, c)$   
if  $m^t = \perp$   
    return false  
if  $\exists (m, r) \in T_G$  s. th.  
 $\text{Enc}(\text{pk}, m'; r) = c$   
    return true  
else return false
```

Fujisaki-Okamoto

Step 5: Getting rigid, OW-PCVA dPKEs

We will now show that the derandomized PKE TPKE matches exactly what the U-transforms want:

We show: if PKE is plain OW secure, then TPKE is

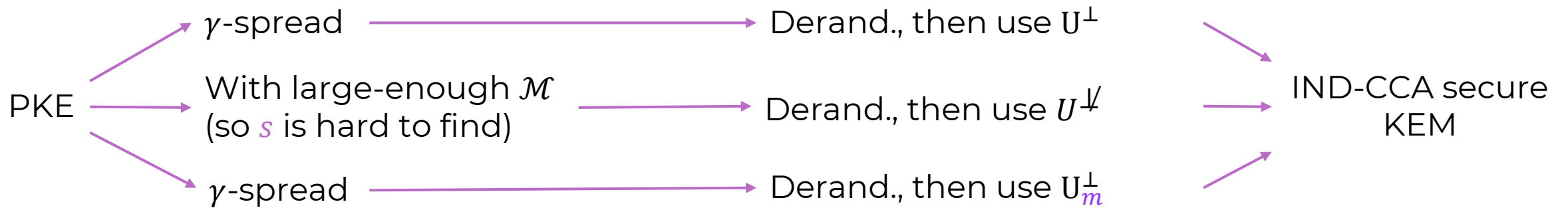
- ✓ rigid;
- ✓ plain OW;
- ✓ OW-PCA, by showing how to simulate PTcheck,
- ✓ OW-VA, by showing how to simulate Valid,
- ✓ thus also OW-PCVA (since both oracles are simulatable).

<u>Enc'(pk, m):</u> $r := G(m)$ $c \leftarrow \text{Enc}(pk, m; r)$ return c	<u>Dec'(sk, c):</u> $m' := \text{Dec}(sk, c)$ if $m' = \perp$ return $\perp$ $c' := \text{Enc}(pk, m'; G(m'))$ if $c' \neq c$ return $\perp$ else return m'
---	---

Fujisaki-Okamoto

What did we learn about U-type transforms?

We can (tightly) build an IND-CCA secure KEM from **probabilistic** PKE as follows:



	Key definition $K := H(m, c)$	$K := H(m)$
Handling of invalid cts		
Respond with $\perp$	$U^\perp$	$U_m^\perp$
Respond with $H(s, c)$, using secret s („implicit rejection“)	$U^\perp$	$U_m^\perp$ (not covered)

Fujisaki-Okamoto

Just for fun: FO in the wild - An actual zoo

decap.sage decode.sage

Datei Bearbeiten Ansicht

```
C0,C1 = C
C0 = list(C0)
C1 = list(C1)
e = decode.decode(C0,Gammaprime,params)
if e == False: return H((0,s,C))
C1prime = H((2,e))
if C1prime != C1: return H((0,s,C))
return H((1,e,C))
```

McEliece Sage code

classic.mceliece.org/spec.html

Algorithm 18 `ML-KEM.Decaps_internal`(dk,c)

Uses the decapsulation key to produce a shared secret key

Input: decapsulation key $dk \in \mathbb{B}^{768k+96}$.

Input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.

Output: shared secret key $K \in \mathbb{B}^{32}$.

- 1: $dk_{PKE} \leftarrow dk[0 : 384k]$ ▷ extract (from KE)
- 2: $ek_{PKE} \leftarrow dk[384k : 768k + 32]$
- 3: $h \leftarrow dk[768k + 32 : 768k + 64]$
- 4: $z \leftarrow dk[768k + 64 : 768k + 96]$
- 5: $m' \leftarrow \text{K-PKE Decrypt}(dk_{PKE}, c)$
- 6: $(K', r') \leftarrow G(m' || h)$

FIPS 203 (ML-KEM)

nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf

```
# Generate pseudorandom values:
seedSE' || k' = SHAKE(pk || u' || salt, len
```

...

```
B = Unpack(b, n, nHat)
```

```
V = S' * B + E"
```

```
C' = V + Encode(u')
```

```
kHat = k' if B' == B" and C == C' else kHat =
ss = SHAKE(c1 || c2 || salt || kHat, lensec)
```

FrodoKEM (IETF spec)

ietf.org/archive/id/draft-longa-cfrg-frodokem-01.txt

All current post-quantum schemes reject “implicitly”.

Some hash c into the key, some don't. Some use additional ‘key confirmation tags’.

Schemes with small message space (Frodo, HQC) additionally use a salt.

Many additionally hash public key-related info (for multi-user security). Some do pre-hashing.

Fujisaki-Okamoto

Just for fun: A lot of recent research on FO

Since FO is important for recent pq standardization efforts, there also is work on if/how...

- ...to rule out quantum attacks.
- ...imperfect correctness of PKEs actually threatens security.
- ...to formally verify the security proofs.
- ...the re-encryption check messes with side-channel security, but is somewhat unavoidable.
- ...one rejection method is better than the other.
- ...to ensure security for many users, many communications.
- ...to prevent implementation footguns.
- ...